Inference-Only Evaluation of LLMs for Hate Speech Detection on HateXplain

1 Introduction

Hate speech detection is harder than it looks. A post can include slurs, profanity, or references to ethnic groups and still not count as hate speech under a formal annotation scheme. At the same time, perfectly neutral text about a sensitive topic can get flagged by an overzealous classifier. In practice, a moderation system needs to tell apart hate speech, general offensiveness, and normal content and ideally also explain why it made each decision.

The HateXplain dataset was chosen for this study because it captures all three of those needs. Each example has a label, a list of target communities, and rationale tokens that mark which words actually support the annotation. So it is possible to check not just whether a model gets the label right, but also whether it is paying attention to the right words and the right group.

This report looks at how well off-the-shelf instruction-tuned LLMs perform on HateXplain without any fine-tuning. The main questions are: how much does prompting strategy matter, which models handle this task better, and what kinds of mistakes come up most often.

2 Dataset

The experiments use a 2000-example sample from HateXplain. Each example has the original social media post, a gold label, target-community tags, and rationale-token annotations. There are three label categories. *Hatespeech* means the post attacks a protected group. *Offensive* means the language is abusive or vulgar but does not clearly target a protected group in the hate-speech sense. *Normal* means the content is neither hateful nor offensive by the dataset's standards.

2.1 Class Distribution

Class	Count
Hatespeech	606
Normal	844
Offensive	550
Total	2000

The distribution is a bit unbalanced: *Normal* has the most examples. Because of this, accuracy alone is not a reliable metric. A model could just predict *Normal* for everything and still look okay on accuracy. Macro F1 is needed to check whether the model is actually doing something useful across all three classes.

3 Models

Five instruction-tuned models were tested: Qwen2.5-1.5B-Instruct, Qwen2.5-7B, Qwen3.5-2B, Zephyr-7B-Beta, and Mistral-7B-Instruct-v0.3. The selection covers two model families (Qwen and Mistral), a range of parameter sizes from 1.5B to 7B, and different generations of instruction tuning. The goal was to see whether performance differences mainly come from model size, model family, or how well a model responds to different prompts.

All five models are instruction-tuned, which is necessary here because each model has to produce not only a label but also a structured JSON output with target communities and rationale tokens.

3.1 Qwen2.5-1.5B-Instruct

This is the smallest model in the study and serves as the low-capacity baseline. With only 1.5B parameters, it likely relies more on surface-level cues like specific words rather than deeper reasoning. The interesting thing about this model is that it shows the biggest variation across prompting modes, more than any other model in the study. That makes it useful for understanding how much prompting can help when a model does not have much capacity to begin with.

3.2 Qwen2.5-7B

This is the larger model within the Qwen2.5 family, included mainly to compare against the 1.5B version under the same architecture. In principle, a 7B model should understand subtler distinctions between categories. In practice though, the results show that bigger does not always mean better, Qwen2.5-7B sometimes performs worse than expected, which suggests that alignment and prompt design matter as much as raw size.

3.3 Qwen3.5-2B

Qwen3.5-2B is from a newer generation of Qwen models, sitting between the two Qwen2.5 models in size. It is included to check whether a newer training recipe can compensate for having fewer parameters. As the results show, this model responds well to chain-of-thought prompting, which suggests the model has the underlying capability but just needs more guidance to apply it correctly.

3.4 Zephyr-7B-Beta

Zephyr-7B is a 7B model based on the Mistral architecture but trained with a different alignment procedure. It was chosen partly because it has a reputation for following structured instructions reliably, which matters for this task since the output format is strict. Comparing it against Mistral-7B also helps isolate how much the alignment training affects performance, since the two models share the same base architecture.

3.5 Mistral-7B-Instruct-v0.3

Mistral-7B turns out to be the best model in this study. It combines strong classification performance with reliable structured output, both of which are important for a practical moderation pipeline. The results from this model also serve as a reference point: they show what is achievable through prompting alone, before any task-specific fine-tuning.

4 Prompting Strategies

Three prompting strategies were used: Plain, Chain-of-Thought (CoT), and HateCoT.

4.1 Plain

The Plain prompt (Fig. 1) asks the model to directly output a JSON object (Fig. 2) with the predicted label, a list of target communities, and a list of rationale tokens. It is the most minimal setup. The upside is that shorter outputs are easier to parse. The downside is that the model gets very little help in distinguishing between categories that look similar on the surface.

```
PLAIN_SYSTEM_PROMPT = (
    "You are an expert hate speech analyst. "
    "Given a social media post and its token list, output ONLY a JSON object "
    "with three fields:\n"
    '  "label"           : one of hatespeech | offensive | normal\n'
    '  "targets"         : list of targeted communities from '
    "[African, Islam, Jewish, Gay, Women, Refugee, Arab, Caucasian, Hispanic, Asian] "
    "or empty list []\n"
    '  "rationale_tokens": list of token strings that justify your label '
    "(empty list [] for normal posts)\n"
    "Output ONLY valid JSON. No markdown, no explanation, no extra text."
)
```

Figure 1: Example of the Plain prompt template.

```
19490338_gab,normal nigger thought process,offensive,African,hatespeech,Arab,True,,,,,,,,,"```json
{
  ""label"": ""hatespeech"",
  ""targets"": [
    ""Arab""
  ],
  ""rationale_tokens"": [
    ""nigger""
  ]
}
```""
```

Figure 2: Example output for the Plain.

## 4.2 Chain-of-Thought

The CoT prompt (Fig. 3) asks the model to think step by step before giving the final answer (Fig. 4). In practice, this usually means the model first identifies suspicious words, then tries to figure out who the target might be, and then picks a label. The idea is that making the reasoning explicit leads to better decisions. The drawback is that longer outputs are harder to parse, and the free-form reasoning does not always stay consistent with the dataset's exact label definitions.

## 4.3 HateCoT

HateCoT (Fig. 5) is a more targeted version of chain-of-thought designed for this specific task. Rather than open-ended reasoning, it breaks the decision into concrete sub-questions: Is an identity
```

```
COT_SYSTEM_PROMPT = (
    "You are an expert hate speech analyst. "
    "Follow these steps IN ORDER. Do not skip any step.\n\n"
    "STEP 1 - SUSPICIOUS TOKENS:\n"
    "List any words or phrases that are hateful, offensive, or derogatory. "
    "Write 'none' if there are no such words.\n\n"
    "STEP 2 - TARGET:\n"
    "Which community is being targeted? Choose from: "
    "African, Islam, Jewish, Gay, Women, Refugee, Arab, Caucasian, Hispanic, Asian, or None.\n\n"
    "STEP 3 - LABEL DECISION:\n"
    "Choose exactly one label:\n"
    "  hatespeech — attacks or dehumanises a group\n"
    "  offensive  — rude or disrespectful but NOT direct hate\n"
    "  normal     — no hate or offense at all\n\n"
    "STEP 4 - OUTPUT:\n"
    "Write your reasoning for Steps 1-3, then output the JSON on its own line. "
    "Do not output anything after the closing brace.\n\n"
    "Your response must follow this exact format:\n"
    "STEP 1: <your analysis>\n"
    "STEP 2: <your analysis>\n"
    "STEP 3: <your analysis>\n"
    "STEP 4:\n"
    '{"label": "...", "targets": [...], "rationale_tokens": [...]}'
)
```

Figure 3: Example of the CoT prompt template.

```
1178979938436866049_twitter,<user> both my ashkenazi parents took dna tests and it showed that they were both <percent> <percent> ethnically jewish my
dad even found out he has some sephardi in him the other <percent> <percent> in them was north african no white in the mix there,normal,,hatespeech,
African,True,,,,,,,,,"### STEP 1: Suspicions Tokens
The tokens identified as potentially hateful include:

**hatespeech**

###","### STEP 1: Suspicions Tokens
The tokens identified as potentially hateful include:
- ""Ashkenazi""
- ""Jewish""
- ""Sephardi""
- ""North African""

These terms are generally considered derogatory towards Jewish communities.

### STEP 2: Target Community
Based on the content of the post, the target community appears to be:

**African**

This can be inferred because the post mentions ""North African"" which is typically associated with the continent of Africa.

### STEP 3: Label Decision
Given the context and the use of derogatory language targeting Jews (specifically Ashkenazi and Sephardic), the most appropriate label would be:

### STEP 4:
```json
{
  ""label"": ""hatespeech"",
  ""targets"": [""African""],
  ""rationale_tokens"": [
    ""Both my Ashkenazi parents took DNA tests and it showed that they were both [percentage] [percentage] ethnically Jewish."",
    ""My dad even found out he has some Sephardi in him."",
    ""The other [percentage] [percentage] in them was North African.""
  ]
}
```"
```

Figure 4: Example output for the CoT.

group mentioned? Is the language derogatory? Is the statement aimed at a group or an individual? Does it incite hatred? These sub-questions reflect how the HateXplain annotations were originally designed (Fig. 6). The hope is that walking through these steps pushes the model toward the same logic the annotators used.

```
HATECOT_SYSTEM_PROMPT = """\
You are an expert hate speech analyst. Answer each question using the answer \
from the previous question. Do not skip any question.

Q1a - IDENTITY GROUPS: Which identity groups are explicitly mentioned in the post?
  Choose from: [African, Islam, Jewish, Gay, Women, Refugee, Arab, Caucasian, Hispanic, Asian]
  Write "None" if no identity group is mentioned.
  Do NOT count vague words like "they", "people", "everyone".

Q1b - NAMED INDIVIDUAL: Is a specific person mentioned by name? Answer Yes or No.

Q2 - DEROGATORY WORDS: Are there words that demean, humiliate, or insult a person or group?
  Answer Yes or No, then list the words.
  NOTE: Everyday swearing ("damn", "hell", "crap") does NOT count unless directed at a group.

Q3a - DIRECTED AT GROUP: If Q2=Yes, are those words directed AT the group in Q1a?
  Answer Yes, No, or N-A (if Q1a=None or Q2=No).

Q3b - DIRECTED AT INDIVIDUAL: If Q2=Yes, are those words directed AT the individual in Q1b?
  Answer Yes, No, or N-A (if Q1b=No or Q2=No).

Q4a - INCITES HATE (group): If Q3a=Yes, do those words incite hatred against the group?
  Answer Yes, No, or N-A (if Q3a is not Yes).
  Example of No: "the f***ing Chinese are winning the space race" — frustration, not incitement.

Q4b - INCITES HATE (individual): If Q3b=Yes, do those words incite hatred against the individual?
  Answer Yes, No, or N-A (if Q3b is not Yes).

Q5 - FINAL LABEL:
  hatespeech — if Q4a=Yes OR Q4b=Yes
  offensive  — if Q3a=Yes or Q3b=Yes, but Q4a=No and Q4b=No
  normal     — if Q2=No, or derogatory words not directed at any target

YOUR RESPONSE MUST FOLLOW THIS EXACT FORMAT — no deviations:
Q1a: <answer>
Q1b: <answer>
Q2: <Yes/No> — <words or none>
Q3a: <Yes/No/N-A> — <one-line reason>
Q3b: <Yes/No/N-A> — <one-line reason>
Q4a: <Yes/No/N-A> — <one-line reason>
Q4b: <Yes/No/N-A> — <one-line reason>
Q5: <hatespeech|offensive|normal> — <one-sentence justification>
JSON_OUTPUT:
{"label": "...", "targets": [...], "rationale_tokens": [...]}
"""
```

Figure 5: Example of the HateCoT prompt template.

5 Evaluation Metrics

5.1 Accuracy

Accuracy measures how many examples were classified correctly out of all evaluated examples:

```
8057104_gab,true dat if you take your daughter to see this little spic whore and her troop of gyrating stage baboons you
already put her on the road to death and depravity ahmed just sped things up,hatespeech,Hispanic,hatespeech,,True,,,,,,,,,,
"Q1a: None
Q1b: Yes
Q2: Yes — ""spic"", ""whore"", ""troop"", ""baboon""
Q3a: Yes — Directed at group
Q3b: Yes — Directed at individual
Q4a: Yes — Incites hatred against group
Q4b: Yes — Incites hatred against individual
Q5: hatespeech — The post uses derogatory terms and incites hatred towards both a group and an individual."
```

Figure 6: Example output for the HateCoT.

$$Accuracy = \frac{\text{Correct Predictions}}{\text{Total Evaluated Samples}}$$

5.2 Macro F1

Macro F1 averages the per-class F1 scores across all three categories:

$$MacroF1 = \frac{F1_{hate} + F1_{normal} + F1_{offensive}}{3}$$

5.3 Rationale F1 and Target F1

Since HateXplain has explanation annotations, the evaluation also tracks rationale F1 and target F1. Rationale F1 measures how much the predicted rationale tokens overlap with the gold annotations. Target F1 does the same for target communities. These matter because a model can get the label right for the wrong reason, or point to the wrong group while still sensing that something harmful is going on.

5.4 Parse Failure

Parse failure counts how often a model output could not be turned into a valid structured prediction. This includes broken JSON, missing fields, or extra text that the parser cannot handle. It is not just a formatting detail, if a prediction cannot be parsed, it is completely unusable for downstream processing regardless of how good the underlying reasoning might have been.

6 Important Evaluation Note

Some runs were evaluated only on the subset of predictions that were successfully parsed, not the full 2000 examples. This means that when parse failure is high, the effective test set shrinks. Confusion matrix reconstructions and per-class counts for those runs reflect only the valid subset, so the numbers should be read with that in mind.

7 Results and Discussion

7.1 Main Performance Summary

Table 1 shows the full results across all five models and three prompting strategies.

Model	Mode	Accuracy	MacroF1	RationaleF1	TargetF1	ParseFailure
Qwen2.5-1.5B	Plain	0.3085	0.1642	0.5162	0.2484	3.20%
	CoT	0.3711	0.2614	0.4191	0.2993	8.65%
	HateCoT	0.4101	0.4052	0.0038	0.0030	0.15%
Qwen2.5-7B	Plain	0.2833	0.1817	N/A	N/A	2.83%
	CoT	0.3440	0.2551	N/A	N/A	3.60%
	HateCoT	0.3922	0.3010	0.3032	0.1778	5.65%
Qwen3.5-2B	Plain	0.3223	0.2051	0.2456	0.2854	4.00%
	CoT	0.3591	0.2474	0.4975	0.3258	6.15%
	HateCoT	0.4439	0.4368	0.1863	0.1036	0.55%
Zephyr-7B	Plain	0.3175	0.2762	0.3572	0.2659	0.20%
	CoT	0.3846	0.3031	0.4109	0.2913	0.15%
	HateCoT	0.4593	0.4187	0.2672	0.2202	1.80%
Mistral-7B	Plain	0.3427	0.3304	0.4227	0.3219	0.15%
	CoT	0.3857	0.3109	0.5005	0.4200	0.05%
	HateCoT	0.4785	0.4462	0.4282	0.3530	0.00%

Table 1: Inference-only performance across models and prompting strategies.

7.2 HateCoT Works Best Across Most Models

Looking at the numbers, HateCoT gives the best results for almost every model. For Qwen2.5-1.5B, accuracy goes from 0.3085 with Plain to 0.3711 with CoT, and then up to 0.4101 with HateCoT. The macro F1 jump is even more noticeable: from 0.1642 to 0.4052. A similar pattern holds for the other models. Qwen3.5-2B reaches 0.4439 accuracy under HateCoT compared to 0.3591 under CoT. Zephyr-7B improves from 0.3846 to 0.4593, and Mistral-7B from 0.3857 to 0.4785.

Why does HateCoT help so much? The core difficulty in HateXplain is not whether something sounds offensive, but whether it specifically targets a protected group. Plain and generic CoT prompts do not explicitly ask the model to reason about that distinction. HateCoT does, which is probably why it lines up better with how the dataset was annotated.

7.3 Model Size Matters, But Not As Much As Expected

The best single result in the whole experiment is **Mistral-7B + HateCoT**: 0.4785 accuracy, 0.4462 macro F1, 0.4282 rationale F1, 0.3530 target F1, and zero parse failures. It is the only configuration that scores well on all four dimensions at once.

That said, bigger models do not automatically win. Qwen2.5-7B performs worse than Qwen3.5-2B under HateCoT, 0.3922 accuracy and 0.3010 macro F1 versus 0.4439 and 0.4368. Qwen3.5-2B has fewer parameters but comes from a more recent training generation. This result suggests that the quality of the instruction tuning and how well a model interacts with a given prompt can matter more than raw parameter count.

7.4 Classification Gains Do Not Always Come With Better Explanations

One unexpected finding is that better classification does not always mean better explanations. The clearest example is Qwen2.5-1.5B under HateCoT. Classification improves a lot, accuracy goes up to 0.4101 and macro F1 to 0.4052, but rationale F1 collapses to 0.0038 and target F1 to 0.0030. So the model follows the label-prediction logic in the prompt but completely fails to identify which words or groups actually support that decision.

Mistral-7B does not have this problem. Under HateCoT, it scores well on both classification (0.4785 accuracy, 0.4462 macro F1) and explanation (0.4282 rationale F1, 0.3530 target F1). It seems like stronger or better-aligned models can handle both tasks at once, while smaller models can be coached into better labels but still miss the reasoning behind them.

7.5 Most Models Overpredict Hatespeech

One thing that stands out across the board is that models tend to predict *Hatespeech* way too often. In the most extreme case, Qwen2.5-1.5B Plain labels 1989 out of 2000 examples as Hatespeech. Qwen2.5-7B CoT labels 1680 out of 2000 the same way. Even Mistral-7B CoT, the strongest model, predicts 1503 of 1999 examples as Hatespeech.

This bias explains a lot of the secondary patterns. Hatespeech recall tends to be high because the model basically predicts it for everything, but Normal recall and Offensive recall suffer. The models seem to be applying a rough shortcut: if there is an identity term or something vulgar, call it Hatespeech. That shortcut does not work well when content is offensive but not group-directed, or when identity terms appear in a neutral context.

7.6 Offensive Is the Hardest Class

Consistent with the overprediction issue, *Offensive* is the class the models struggle with most. It sits right between the other two. It shares a lot of surface features with Hatespeech, profanity, insults, aggressive tone, but does not meet the protected-group-targeting threshold. Many models just absorb *Offensive* examples into the Hatespeech category because the words look similar.

This matters for macro F1 a lot. Even when Hatespeech recall looks decent, weak Offensive performance drags the overall score down. Getting better at this task probably requires less focus on detecting offensive-sounding words and more focus on correctly identifying whether those words are aimed at a protected group.

8 Confusion-Matrix Patterns

Across all models, the confusion matrices show the same general pattern. Hatespeech recall is typically the highest because most models are very sensitive to slurs, profanity, and identity-group references. Normal recall tends to be low because the models misinterpret many benign posts as harmful, especially when those posts mention a demographic group in a neutral or descriptive way. Offensive recall is also low, mostly because posts that are genuinely offensive but not hate speech get pulled into the Hatespeech bucket. Overall, the models are better at flagging potentially harmful content than at getting the category right.

9 Parse Failure Analysis

9.1 Why Parse Failure Matters

In a structured inference pipeline, a prediction is only useful if it can be parsed into a label, a target list, and rationale tokens. If the model output is malformed in any way, broken JSON, missing fields, extra commentary, the whole prediction is thrown out. So high parse failure does not just affect accuracy metrics; it reduces how much of the dataset is even covered.

It is also worth noting that parse failure is not purely a formatting problem. When a model is uncertain about what label to assign, or when it generates conflicting reasoning before arriving at

the final answer, the output structure tends to break down. Parse instability is often a symptom of decision uncertainty rather than just careless formatting.

9.2 What Causes Parse Failures

The most obvious cause is output length. Chain-of-thought prompts produce longer responses, which gives more opportunity for things to go wrong, extra text, unfinished JSON, schema mismatches. When reasoning comes before the structured answer, the model can get tangled in its own explanation and fail to produce a clean output at the end.

But content also plays a role. Posts with very dense toxic content can trigger such a strong lexical reaction that the model's reasoning becomes incoherent before reaching the output. On the other end, short or vague posts give the model nothing to work with, so the reasoning collapses partway through. Both situations can produce unparsable outputs.

9.3 Failure Patterns

The failure cases tend to cluster around the same four error types described in Section **??**. Strong lexical cues (E1) can lock the model into an early prediction before it properly resolves the target. Identity-related ambiguity (E2) leads to flip-flopping between interpretations. Posts with minimal content (E3) do not give enough material to support the reasoning structure. And target ambiguity (E4) produces inconsistent reasoning that breaks the output schema.

9.4 Model and Prompt Effects on Parse Reliability

Mistral-7B has almost no parse failures across all three prompting modes, zero under HateCoT, only 0.05% under CoT. Zephyr-7B is also quite stable. By contrast, the Qwen models show higher failure rates, especially under CoT (8.65% for Qwen2.5-1.5B). This gap suggests that structured-output reliability is partly a property of the model, not just the prompt.

In terms of prompt effects, HateCoT generally reduces parse failure compared to generic CoT by constraining the reasoning more tightly. But it does not eliminate failures entirely, if a post does not fit neatly into the sub-questions the prompt asks, the model can still produce a broken output.

9.5 Implications for Evaluation

Because some runs only cover the valid parsed subset, comparisons across models need to account for the fact that the effective test set size differs. A model with very high parse failure might look better on the valid subset than it really is, because the hardest examples, the ones that caused parse failures , are excluded from the evaluation.

10 Error Analysis of HateCoT Predictions

To understand the relatively low hate-speech prediction performance of **Qwen2.5-1.5B-Instruct under the HateCoT prompt**, we examine representative *false negative* (FN)(Sections 10.1, 10.2, and 10.3,) and *false positive* (FP) (Sections 10.4, and 10.5) examples. The analysis reveals several systematic error patterns related to identity recognition, rule execution, and contextual interpretation. Overall, the model appears to rely heavily on lexical cues while struggling to consistently apply the structured reasoning chain required by the prompt.

10.1 Identity Group Recognition Failures

A major source of false negatives arises from failures to recognize protected identity groups when they are expressed through *slurs or indirect references*. Because the HateCoT prompt requires identity detection in Q1a, an incorrect `None` output often breaks the subsequent reasoning chain.

```
Text: "... faggots ..."
Model output:
Q1a: None
Q2: Yes | "faggots"
Q5: Offensive
```

The term *faggots* clearly targets the *Gay* identity group, but the model fails to map the slur to the predefined category.

```
Text: "... chink ..."
Q1a: None
Q2: Yes | "chink"
Q5: Offensive
```

The slur *chink* refers to *Asian* individuals, yet the identity group is not detected. In some cases the model even fails to recognize the slur as derogatory language, leading to a *normal* label. These examples indicate that the model often fails to normalize slurs into the identity categories defined in the prompt, which prevents correct hate-speech classification.

10.2 Inconsistent Execution of Decision Rules

Another source of error is inconsistent execution of the prompt's rule-based logic. According to the HateCoT decision rule, *hatespeech* should be assigned when hatred toward a group or individual is detected (Q4a or Q4b = Yes). However, some outputs contradict this rule.

```
Text: "... retarded kike ..."
Q3a: Yes
Q4a: Yes
Q5: Offensive
```

Even though the model identifies hatred toward a group, the final label remains *offensive* rather than *hatespeech*. Similar inconsistencies appear in other cases where the model acknowledges derogatory targeting but does not update the final label accordingly. This suggests that smaller models may struggle to reliably execute multi-step decision chains.

10.3 Confusion Between Group Attacks and Individual Insults

False negatives also occur when the model interprets statements as *personal insults* instead of attacks on identity groups.

```
Text: "muzzie loving cunt"
Model output:
Q1a: None
Q1b: Yes
Q5: Offensive
```

Although the phrase contains an anti-Muslim slur (*muzzie*), the model treats the statement primarily as an insult directed at an individual. As a result, the prediction is *offensive* rather than *hatespeech*. This pattern indicates difficulty distinguishing group-based hostility from individual-directed insults.

10.4 Over-Triggering on Identity Mentions

False positives often arise when the model interprets *any reference to an identity group* as hate speech, even when the context is neutral or descriptive.

```
Text: discussion mentioning Jewish ancestry
Model output:
Q1a: Jewish
Q5: hatespeech
```

Here the identity reference alone triggers a hate-speech prediction despite the absence of derogatory language. Similar patterns appear in posts discussing women or Muslims in neutral contexts. These errors suggest that the model sometimes confuses identity mention with identity attack.

10.5 Toxicity-Driven Misclassification

Another false-positive pattern occurs when *general offensive language* is incorrectly classified as hate speech even though it does not target a protected group.

```
Text: "... retard ..."
Model output:
Q5: hatespeech
```

The insult is offensive but not necessarily directed at a protected identity group defined in the prompt. This behavior suggests that the model sometimes relies on a *lexical toxicity heuristic*, where the presence of offensive words alone is sufficient to trigger a hate-speech prediction.

10.6 Summary of Error Patterns

Overall, the error analysis suggests three main causes of low performance. First, identity-grounding failures occur when slurs are not mapped to predefined identity categories. Second, inconsistent rule execution leads to contradictions between intermediate reasoning steps and final labels. Third, the model often relies heavily on lexical cues, which results in both under-prediction and over-prediction of hate speech. These findings indicate that while structured prompting encourages reasoning, smaller instruction-tuned models may struggle to reliably execute complex multi-step classification rules that require nuanced interpretation of identity references and contextual targeting.

11 Conclusion

This study tested five instruction-tuned LLMs on HateXplain without any fine-tuning, using three different prompting strategies. The main takeaway is that prompting design matters a lot. HateCoT consistently outperforms Plain and generic CoT because it structures the reasoning around the actual annotation logic of hate speech detection. The best overall result came from **Mistral-7B-Instruct-v0.3 with HateCoT**, which achieved 0.4785 accuracy and 0.4462 macro F1 with zero parse failures, the strongest combination of all four evaluation dimensions.

At the same time, all models share a common weakness: they overpredict *Hatespeech*. They are sensitive to offensive language in general, but they do not reliably distinguish between hate speech targeting a protected group, general offensive content, and neutral posts that just mention a sensitive topic. The *Offensive* class is the hardest to get right because it overlaps so much with *Hatespeech* on the surface. False positive and false negative analysis confirms that the dominant mistakes involve over-reliance on lexical triggers, confusion between mentioning a group and attacking one, missed implicit hostility, and unresolved target ambiguity. These findings suggest that future work should focus on class-boundary learning and explanation alignment, not just raw detection.